

SchemaForge

Rehearse PostgreSQL migrations, prove what happened, stop before production
TrueFoundry × Polaris “Agents That Act” Hackathon · Complete Project Summary · September 2026

The model proposes; tools measure; policy constrains; a human decides.

1 · The problem

Schema migrations are simultaneously the most attractive and the most dangerous job to hand to an AI agent. A model can write plausible DDL in seconds, but *plausible* is not *safe*. A migration that survives code review still fails against real data: a NOT NULL constraint collides with rows nobody knew were empty, a type change silently truncates, an ACCESS EXCLUSIVE lock stalls a hot table, or a dropped column takes its data with it. Three things are missing from the naive “agent writes SQL and runs it” loop:

- **Evidence.** Nothing has actually executed against representative data, so every claim about duration, locks and row impact is a guess dressed as a fact.
- **Reversibility.** A rollback script that has never been run is a hypothesis, not a plan.
- **A boundary the agent cannot cross.** “Ask before doing something irreversible” is a prompt instruction, and prompt instructions are advice, not enforcement.

SchemaForge treats all three as engineering problems rather than prompting problems.

2 · What it is

A TrueForge agent that converts a natural-language database change into an evidence-backed decision. It inspects a live PostgreSQL 16 target, maps the dependency blast radius, executes the generated SQL inside a rollback-only shadow sandbox, checks explicit assertions and rollback equivalence, and then **freezes** with a decision packet. Nothing reaches production without a cryptographically signed approval bound to that exact SQL and that exact assertion set, plus a second Allow/Deny gate inside TrueForge. The interesting output is often the refusal.

3 · Architecture

Two role-separated MCP servers (TypeScript, Streamable HTTP) attach to one TrueForge agent. The split is structural: configuration validation refuses to boot a process holding a credential it should not have.

Component	Responsibility and credentials
schemaforge-core :3100	Five read/rehearse tools: schema inspection, bounded read-only queries, dependency analysis, shadow rehearsal, post-apply verification. Holds only <code>sf_reader</code> (production SELECT) and <code>sf_shadow</code> ; cannot write production or mint approvals.
schemaforge-executor :3101	Exactly one tool, <code>execute_migration</code> , published with <code>destructiveHint: true</code> . Holds the <code>sf_executor</code> write role (not a superuser) and the approval key; has no shadow access.
Approval CLI	<code>npm run approve</code> runs in the operator’s terminal with only the signing secret and TTL — no database URL. The key never enters the agent loop.
PostgreSQL :5433 / :5434	Production and shadow, in Docker Compose, seeded from the <i>same</i> deterministic file: 500 users, exactly 14 NULL emails, a three-row duplicate-email group, boundary foreign keys, precision and zero-quantity edges.
Execution ledger	Table in production recording every attempt. <code>sf_executor</code> may INSERT and update only four status columns; the nonce is immutable and migration SQL may not reference the table.

Correctness mechanics. Every tool call runs on one physical connection, so BEGIN, timeouts, work and COMMIT are never split across pooled sessions. Reads and fingerprints use REPEATABLE READ, so one hash cannot blend two catalog snapshots. Rehearsal runs inside a transaction that is *only ever rolled back*, with the rollback confirmed rather than assumed and a poisoned connection destroyed instead of reused. Production applies serialize behind an advisory lock under statement and lock timeouts, so a blocked migration fails fast instead of queueing traffic.

4 · The nine-stage workflow

1. **Parse intent** — restate the objective and success condition; ask a structured question rather than guess.
2. **Inspect** — catalog, version, extensions, and a stable baseline SHA-256 fingerprint (row-count estimates deliberately excluded).
3. **Analyze dependencies** — foreign keys, views and materialized views, functions, triggers, indexes, RLS policies, owned sequences.
4. **Synthesize** — byte-stable forward SQL plus honest rollback SQL, each statement classified for transaction semantics.
5. **Rehearse** — execute on the shadow session; capture measured duration, lock snapshot, PostgreSQL notices, exact row deltas, and post/rollback fingerprints.
6. **Verify** — run named assertions with declared expectations; confirm rollback restores the baseline hash exactly.
7. **Assess** — deterministic APPLY / REVIEW / DO_NOT_APPLY, computed in code.
8. **Freeze** — publish the decision packet and stop; no production call in the same turn.
9. **Execute** — only on a new human message carrying a valid token, and only through the TrueForge approval gate.

Evidence is labelled: values returned by a tool are [OBSERVED]; catalog statistics and model inference are [ESTIMATED]. A lock snapshot is never reported as a lock duration.

The decision packet is the artifact the human actually reads, and it has a fixed shape: objective; baseline fingerprint, affected objects and dependency counts; the exact forward and rollback SQL with each statement’s transaction class; rehearsal evidence (rehearsal ID, measured duration, lock snapshot, notices, row deltas, and the baseline/post/rollback fingerprints); every named assertion with its actual result; a risk level with its evidence-based factors; and the verdict with rationale. Any field the tools could not produce is written UNAVAILABLE rather than filled with plausible prose — an honest inability to prove safety is a reason to stop, not a gap to paper over.

5 · Where it stops

The verdict is computed by rule code, not model opinion. The model may explain it; it cannot overturn it.

Evidence	Verdict
SQL fails policy (prohibited, unsupported, mixed or non-transactional)	rejected before rehearsal
Forward SQL fails, or any named assertion fails	DO_NOT_APPLY
Rollback ran but did not restore the baseline fingerprint	DO_NOT_APPLY
No rollback supplied, or the plan is classified destructive	REVIEW
Rehearsal, assertions and rollback equivalence all verified	APPLY

A production apply is refused with a distinct machine-readable code, each covered by an automated test:

Code	Cause
SIGNATURE_INVALID / TOKEN_MALFORMED	Forged, tampered or unparseable approval
HASH_MISMATCH / ASSERTIONS_MISMATCH	SQL or the assertion set edited after the human approved it
TOKEN_EXPIRED / TOKEN_NOT_YET_VALID / TTL_EXCEEDED	Outside the five-minute window, or an over-long TTL
REPLAY_DETECTED	Nonce already consumed in the production ledger
SCHEMA_DRIFT	Production no longer matches the rehearsed baseline
POSTCONDITION_FAILED	Post-apply fingerprint or an approved assertion failed; transaction rolled back
SQL_POLICY_REJECTED	Statement fails the executor's independent policy re-check

Refused unconditionally, even with a valid signature: DROP TABLE, TRUNCATE, DROP DATABASE/SCHEMA, role and system administration, GRANT/REVOKE, COPY ... PROGRAM, procedural blocks, all DML, any reference to the ledger, and anything outside the canonical fingerprint model. The policy fails *closed*: ambiguous input, such as backslash-escaped literals it cannot lex with certainty, is rejected rather than guessed at.

6 · How it is used in TrueForge

TrueForge supplies the agent loop, session state, sandbox and human checkpoints; SchemaForge supplies the tools and the boundary.

1. Run the harness (`npx @truefoundry/trueforge@0.2.1`) and connect a model under **Settings** → **Models**.
2. Register two remote MCP connectors: `schemaforge-core` at `http://127.0.0.1:3100/mcp` and `schemaforge-executor` at `:3101/mcp`.
3. Configure a Daytona sandbox provider, used for the agent's own helper code and skills.
4. Create the agent, pasting `trueforge-config/system-prompt.md` into Instructions; `agent.yaml` mirrors the remaining fields.
5. Attach both connectors and require approval for `execute_migration`.

Two independent human boundaries result. TrueForge gates tools the MCP server marks destructive — its documented default is `["@destructive"]`, and the tool is additionally named explicitly — so the chat pauses on an Allow/Deny card showing the tool and its arguments. Separately, the executor will not act on an Allow alone: it demands the signed token the operator minted in their own terminal after reading the decision packet. Compromising the agent loop is therefore not sufficient to change production. Two prompts make the story visible: asking for `users.email` to become `NOT NULL UNIQUE` surfaces the 14 `NULL`s and the duplicate group and ends in `DO_NOT_APPLY`, production untouched; adding a nullable `phone` column rehearses cleanly, proves its rollback, pauses twice, applies atomically, then rejects a replay of the same token.

7 · Real-world application

The immediate use is database change management for teams without a dedicated DBA: the agent does the archaeology — which views, constraints and foreign keys are implicated, how many rows actually violate the new invariant — and hands over a reviewable packet instead of a diff. It fits naturally as a migration gate in CI/CD, and the ledger doubles as an audit trail: every attempt, approval, rejection code and post-apply fingerprint is recorded in the target database, which is what a regulated environment or a post-incident review asks for.

The transferable part is the pattern, not the SQL. Any domain with cheap analysis and expensive mistakes can reuse it: rehearse in a disposable copy, gather measured evidence, compute the verdict in code, bind a short-lived signed approval to one exact action, verify postconditions inside the same transaction, and record the outcome. Cloud teardown, access revocation, release publishing and infrastructure runbooks share that shape — the agent is trusted to investigate, never to decide.

8 · Known limitations

Stated plainly, because a safety tool that overstates its reach is the problem it claims to solve.

- **PostgreSQL and the public schema only.** One database per target; no MySQL, no cross-schema or cross-database migrations.
- **Transactional schema DDL only.** CREATE INDEX CONCURRENTLY, REINDEX CONCURRENTLY and VACUUM cannot be made atomic, so they are refused rather than pretended to be reversible; they need a separately designed recovery workflow.
- **No data migrations.** All DML is rejected. Backfills, deduplication and the remediation that would make the NOT NULL example succeed are out of scope — the executor proves schema state, not data equivalence.
- **The fingerprint defines the frontier.** It covers tables, columns with full type modifiers, constraints, indexes, views and extensions. What it cannot represent — function bodies, trigger state, ownership, partition topology, storage parameters — is excluded from the allowlist rather than silently unverified.
- **The SQL policy is a conservative lexer with an allowlist, not a complete PostgreSQL parser.** It is designed to under-approximate: the database's own least-privilege roles remain the backstop.
- **Rehearsal measures a seeded copy, not production scale.** Timings and lock observations are directionally useful, not capacity planning, and the lock reading is a point-in-time snapshot.
- **Local, single-tenant scope.** One approval key, one target, no multi-tenant routing; the bundled credentials are demo values for a local Compose stack, and the harness itself runs in TrueForge's localhost mode.
- **Verification status.** Types, build, 32 security unit tests and 7 live MCP transport/boot tests pass locally; the 25 PostgreSQL scenarios run in CI against fresh PostgreSQL 16 containers, and were skipped on the authoring machine because Docker was not running.